

Escalonamento de processos

Samuel Francisco Ferrigo

Conceito

O escalonamento é necessário para que a CPU decida a ordenação da execução dos processos

O escalonador é o responsável por realizar o escalonamento, a partir do uso de um algoritmo de escalonamento

Desafios do escalonamento

Permitir a execução de todos os processos

Diminuir o impacto da interrupção, pois fazer a troca de contexto é complexa

- Troca do modo usuário para o modo núcleo
- Salvar o estado atual do processo na tabela de processos
- Selecionar novo processo
- Carregar o estado atual do novo processo

Gerenciamento de processo	Gerenciamento de memória	Gerenciamento de arquivo
Registros	Ponteiro para informações sobre o segmento de texto	Diretório-raiz
Contador de programa		Diretório de trabalho
Palavra de estado do programa	Ponteiro para informações sobre o segmento de texto	Descritores de arquivo
Ponteiro da pilha		ID do usuário
Estado do processo	Ponteiro para informações sobre o segmento de texto	ID do grupo
Prioridade		
Parâmetros de escalonamento		
ID do processo		
Processo pai		
Grupo de processo		
Sinais		
Momento em que um processo foi iniciado		
Tempo de CPU usado		
Tempo de CPU do processo filho		
Tempo do alarme seguinte		

Relembrando o Bloco de Controle do Processo (Process Control Block - PCB), que contém os dados de um processo que são salvos na tabela de processos,...

Comportamento de um processo

Os processos podem possuir dois tipos de comportamento, conforme suas limitações:

- Limitado pela CPU (CPU-bound), quando apresentam longo uso (surto) de CPU e poucas esperas de E/S
- Limitado pela E/S (I/O-bound), quando apresentam curto uso de CPU e frequentes esperas por uso de E/S

Qual dos dois comportamentos impacta mais no desempenho de um computador?

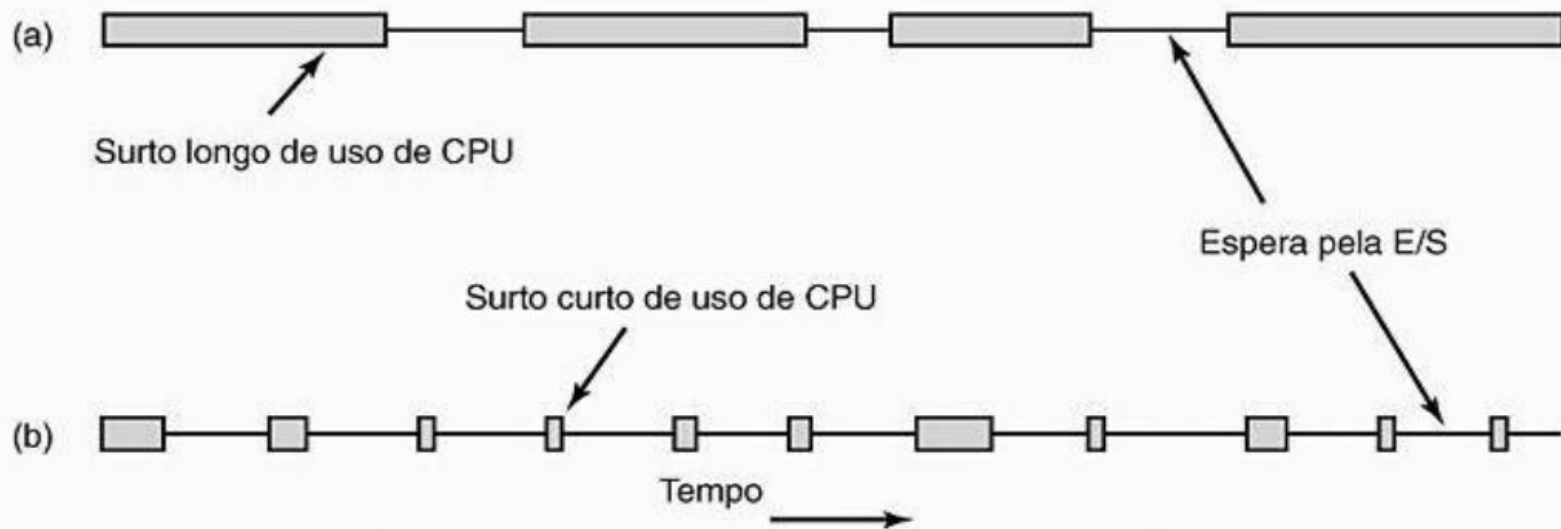


Figura 2.31 Usos de surtos de CPU se alternam com períodos de espera por E/S. (a) Um processo orientado à CPU. (b) Um processo orientado à E/S.

Comportamento do processo. Tanenbaum (2009)

Quando precisamos de um escalonador?

Ao iniciar um novo processo

Ao término de um novo processo

Ao bloqueio de um processo

- Ação de E/S
- Semáforo

Interrupção

Preempção vs não-preempção

Não preemptivo

- O escalonador aguarda o processo terminar sua execução ou ser bloqueado

Preemptivo

- Há fixação de seu tempo máximo de execução, chamado de **quantum**, suspendendo o processo caso o tempo esgote
- Esse tempo máximo de execução é gerenciado pelo SO por meio de uma interrupção de relógio
 - Gerada por um temporizador interno ou um relógio de sistema
 - Usada para manter a precisão do tempo do sistema

Quantum

A definição do quantum é extremamente importante

Se o quantum for muito curto, a CPU gastará boa parte do tempo fazendo a troca de contexto

Se o quantum for muito longo, pode ser que a resposta do sistema ao usuário se torne muito lenta

O ideal é que o sistema realize o menor número possível de troca de contextos sem tornar a resposta muito lenta

Objetivos do escalonamento

Os objetivos do escalonamento de processos dependem da finalidade ou do ambiente onde são executados

- Sistemas em lote
 - Folha de pagamento, previsão do tempo, etc
- Sistemas Interativos
 - Usuário interage constantemente
- Sistemas de tempo real
 - Aviação, militar, etc

Todos os sistemas

Justiça — dar a cada processo uma porção justa da CPU

Aplicação da política — verificar se a política estabelecida é cumprida

Equilíbrio — manter ocupadas todas as partes do sistema

Sistemas em lote

Vazão (*throughput*) — maximizar o número de tarefas por hora

Tempo de retorno — minimizar o tempo entre a submissão e o término

Utilização de CPU — manter a CPU ocupada o tempo todo

Sistemas interativos

Tempo de resposta — responder rapidamente às requisições

Proporcionalidade — satisfazer às expectativas dos usuários

Sistemas de tempo real

Cumprimento dos prazos — evitar a perda de dados

Previsibilidade — evitar a degradação da qualidade em sistemas multimídia

Alguns dos objetivos do escalonamento

Escalonamento em sistemas em lote

Primeiro a chegar, primeiro a ser servido (FCFS - First come, first serve)

Tarefa mais curta primeiro (SJF - Shortest Job First)

Próximo de menor tempo restante (SRTN - Shortest Remaining Time Next)

FCFS (First Come, First Serve - Primeiro a chegar, primeiro a ser servido)

Não preemptivo

É o mais simples de ser executado

Trabalha com uma única fila

Os processos criados ou desbloqueados vão para o fim da fila

Ruim para trabalhar com processos voltados à E/S

SJF (Shortest Job First - Tarefa mais curta primeiro)

Não preemptivo

Possui uma fila de execução que ordena a execução dos processos do mais curto para o mais longo

Pressupõe que os tempos de execução de todos os processos a serem executados sejam conhecidos

Reduz o tempo médio de retorno, ou seja, a média de tempo de execução de cada processo

Medidas de desempenho de um escalonador

TE = Tempo de espera do processo. Quanto tempo o processo esteve na fila aguardando para usar o processador

TR = Tempo de retorno do processo, ou seja, quanto tempo um processo levou para ser totalmente executado desde a sua criação

TEM = soma de todos os tempos de espera dos processos / número de processos

TRM = soma de todos os tempos de retorno dos processos / número de processos

Exemplo

Considere os seguintes processos:

- A = 8 minutos
- B = 4 minutos
- C = 6 minutos
- D = 2 minutos

Qual será a ordenação de execução dos processos com o algoritmo FCFS e SJF?

Qual será o tempo de retorno médio (TRM) dos processos em cada um dos algoritmos?

Qual o será o tempo de espera médio (TEM) das tarefas em cada um dos algoritmos?

SRTN (Shortest Remaining Time Next - Próximo com o tempo restante mais curto)

É uma variação preemptiva do SJF

A escolha do próximo processo a ser executado é definida com base no menor tempo de execução restante

Quando chega uma nova tarefa, seu tempo de execução é comparada com a tarefa em execução.

- Se a primeira for menor, o processo em execução é suspenso e a nova tarefa é carregada

Escalonamento em sistemas interativos

Chaveamento circular (round-robin)

Prioridades

Filas múltiplas

Próximo processo mais curto (SPN - Shortest Process Next)

Escalonamento garantido

Loteria

Fração Justa (Fair-share)

Round-robin (Escalonamento circular)

Preemptivo

Atribui-se um quantum ao processo

Ao terminar o quantum, se o processo não concluiu sua execução, há uma troca de contexto

Se houver bloqueio ou o processo termina antes do quantum, o escalonador carrega outro processo `CPU

Prioridades

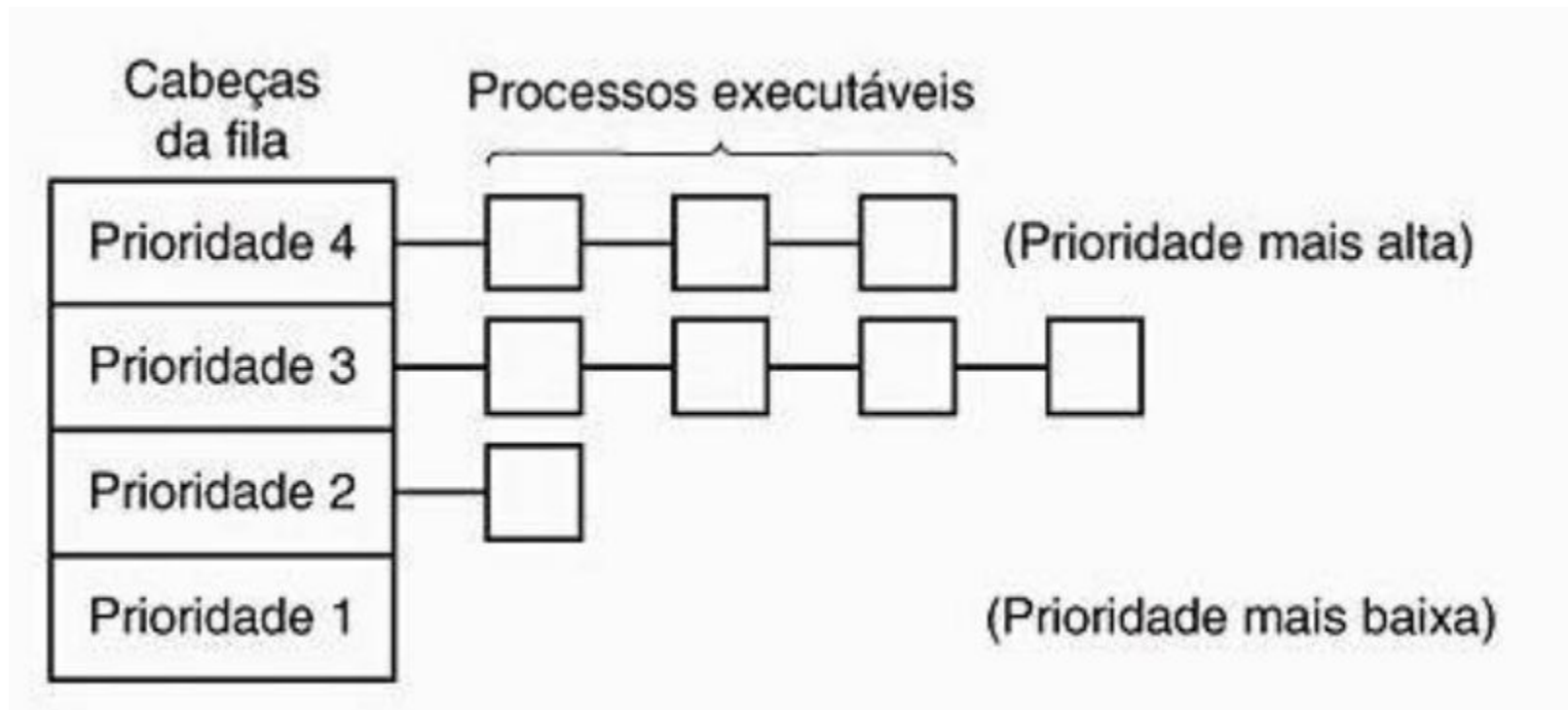
Normalmente preemptivo

A cada processo é atribuída uma prioridade

A ordem de execução segue o nível de prioridades

Para que um processo não execute indefinidamente, pode-se reduzir a sua prioridade a cada interrupção de relógio ou um quantum máximo de execução

No Linux pode-se atribuir uma prioridade pelo comando `nice`



Classes de prioridade

Filas múltiplas

Preemptivo

Nas filas múltiplas, a quantidade de quantums destinados a cada processo depende da sua prioridade

Quanto maior a prioridade, maior o número de quantums que o processo terá direito na sua vez

Próximo processo mais curto (SPN - Shortest Process Next)

Preemptivo

Calcula a estimativa de tempo restante com base no comportamento anterior dos processos

- Técnica conhecida como envelhecimento (aging)
 - Cálculo: $T_{\text{NOVO}} = \alpha \cdot T_{\text{ANTERIOR}} + \alpha \cdot T_{\text{ATUAL}}$
 - onde α é uma constante que representa o peso do histórico anterior. Normalmente usa-se 0,5

Escalonamento garantido

Preemptivo

O uso da CPU é compartilhado igualmente entre o número de processos ou de usuários conectados

O escalonador calcula continuamente se o uso da CPU está dentro do tempo estimado (valor 1)

- Se o valor estiver abaixo de 1, mais tempo será destinado ao processo
- Se estiver acima, menos tempo será destinado

Escalonamento garantido

Exemplo

Processo	Tempo de CPU recebido (%)	Tempo de CPU ideal (%)	Razão	Decisão do escalonador
P1	20	25	0.8	Aumenta prioridade
P2	10	25	0.4	Aumenta prioridade
P3	30	25	1.2	Diminui prioridade
P4	40	25	1.6	Diminui prioridade

Loteria

Preemptivo

Trabalha com a ideia de distribuir bilhetes aos processos

O escalonador sorteia um bilhete, e o processo que o detém usa a CPU

Processos com maior prioridade ganham mais bilhetes

Os processos podem “trocar” bilhetes

- No caso de um bloqueio, um processo pode doar os bilhetes a outro, que devolve-os caso o processo bloqueado seja retomado

Esse sistema torna o escalonamento responsivo

Fair Share (Fração Justa)

Preemptivo

O tempo de uso da CPU é dividido igualmente entre o número de usuários, independentemente da quantidade de processos executados por cada usuário

Escalonamento em sistemas de tempo real

Um sistema de tempo real somente pode ser escalonável se todos os eventos periódicos de todos os processos a serem executados não ocuparem 100% da CPU

Divididos em críticos e não-críticos

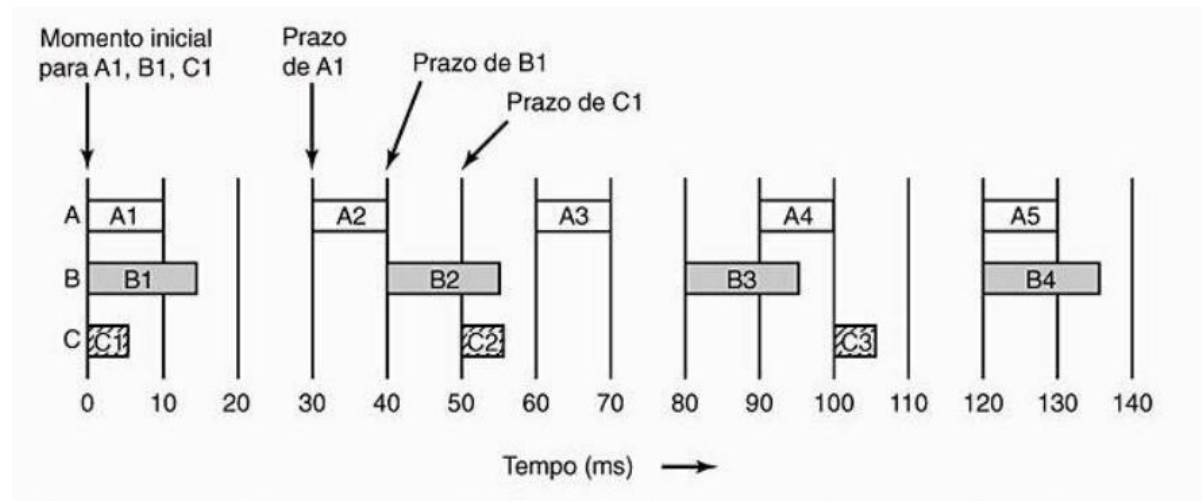
- Nos críticos, os prazos devem ser cumpridos
- Nos não-críticos, o eventual não cumprimento dos prazos é tolerável

Podem também ser periódicos e aperiódicos

- Nos periódicos, ocorrem em prazos regulares
- Nos aperiódicos, ocorrem em intervalos imprevisíveis

Escalonamento em sistemas de tempo real

No exemplo da figura ao lado, os processos podem ser escalonados?



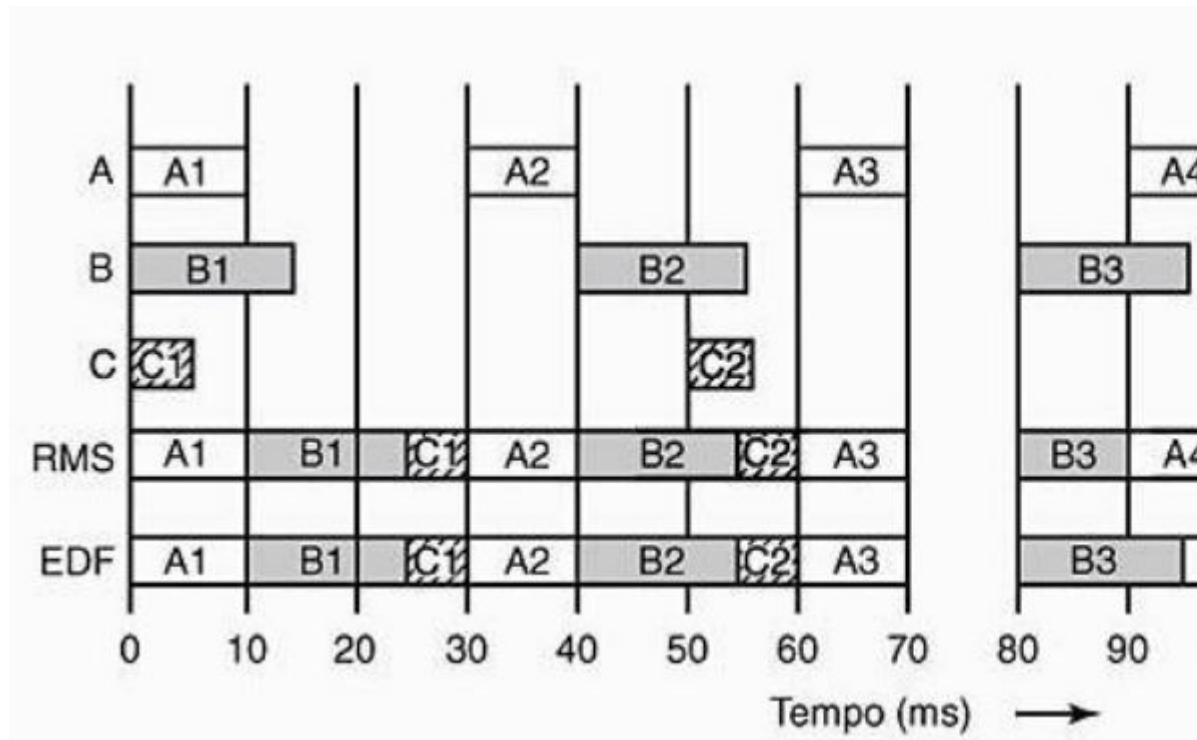
Algoritmos de escalonamento em sistemas de tempo real

Por taxa monotômica (RMS - Rate monotomis Schedule)

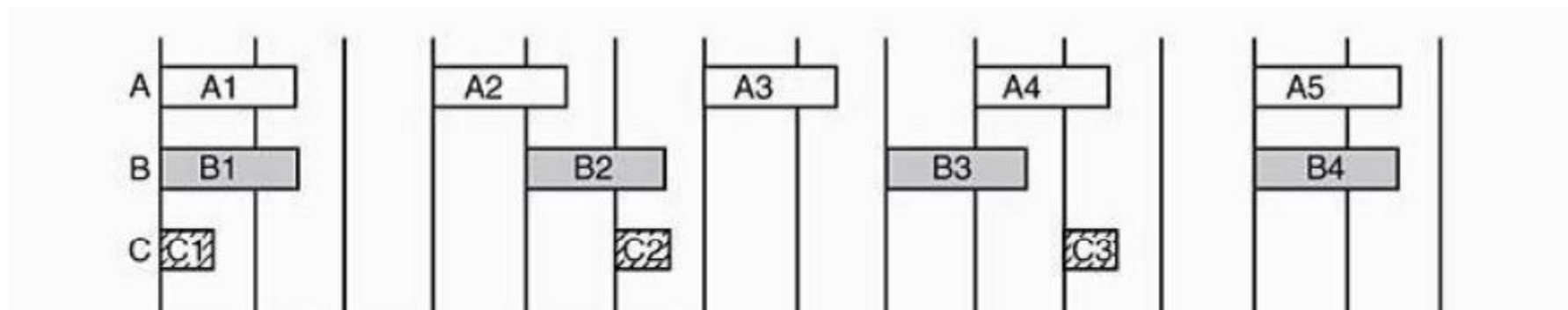
- Preemptivo e periódico
- A prioridade de execução é definida conforme a sua frequência de execução
- Funciona bem em casos de ocupação de até 78% da CPU

Prazo final primeiro (EDF - Earliest deadline first)

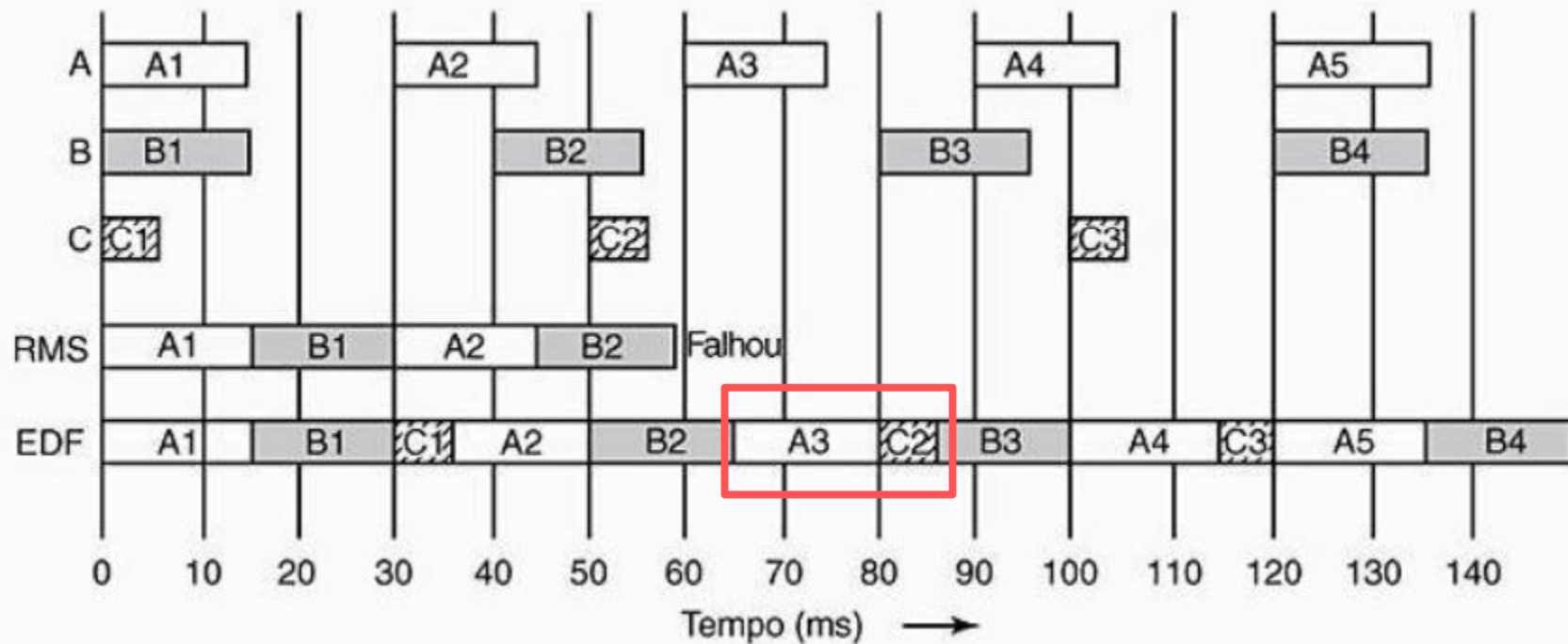
- Preemptivo
- Executa os processos conforme o prazo final de execução esteja mais próximo de vencer



Neste exemplo, ambos algoritmos atendem



E neste caso, o que ocorreria com cada um dos algoritmos?



Já neste, somente o EDF funciona adequadamente

Política de escalonamento

Outra forma de escalonar processos é através de políticas, onde os processos fornecem parâmetros que auxiliam o escalonador a tomar a decisão de escalonamento

- Um exemplo é a política que define a prioridade

Escalonamento em threads

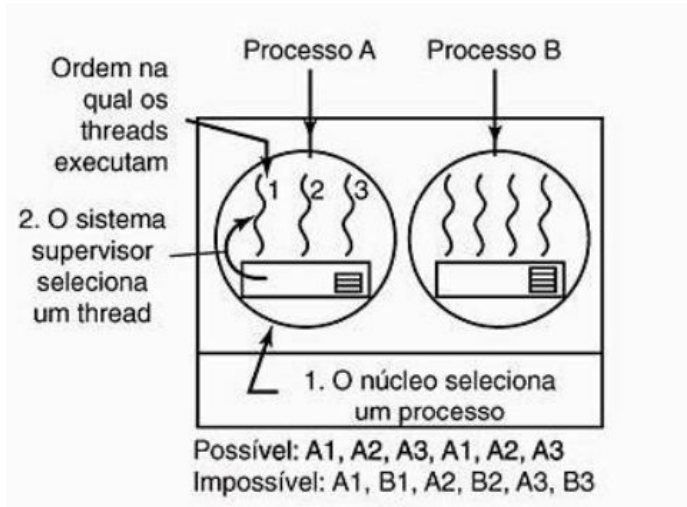
Como as threads compartilham uma mesma memória, o escalonamento entre threads de um mesmo processo é muito menos custoso

Dependendo da região onde estiver sendo executada, o escalonador poderá ou interferir na ordenação de execução das threads entre diferentes processos

- Se a thread estiver sendo executada em modo usuário, ele não poderá interagir entre threads de diferentes processos
- Se estiver em modo núcleo, o escalonador poderá intercalar entre threads de diferentes processos

Escalonamento de threads

Threads de usuário



Threads de núcleo

